

Lightweight RPC for Kotlin Multiplatform

by

Aleksandr Stupnikov

Master Thesis in Computer Science and Software Engineering

Submission: May 5, 2026

Supervisor: Prof. V. Confused

Statutory Declaration

Family Name, Given/First Name	Stupnikov, Aleksandr
Matriculation number	00101010
Kind of thesis submitted	Master Thesis

English: Declaration of Authorship

I hereby declare that the thesis submitted was created and written solely by myself without any external support. Any sources, direct or indirect, are marked as such. I am aware of the fact that the contents of the thesis in digital form may be revised with regard to usage of unauthorized aid as well as whether the whole or parts of it may be identified as plagiarism. I do agree my work to be entered into a database for it to be compared with existing sources, where it will remain in order to enable further comparisons with future theses. This does not grant any rights of reproduction and usage, however.

This document was neither presented to any other examination board nor has it been published.

German: Erklärung der Autorenschaft (Urheberschaft)

Ich erkläre hiermit, dass die vorliegende Arbeit ohne fremde Hilfe ausschließlich von mir erstellt und geschrieben worden ist. Jedwede verwendeten Quellen, direkter oder indirekter Art, sind als solche kenntlich gemacht worden. Mir ist die Tatsache bewusst, dass der Inhalt der Thesis in digitaler Form geprüft werden kann im Hinblick darauf, ob es sich ganz oder in Teilen um ein Plagiat handelt. Ich bin damit einverstanden, dass meine Arbeit in einer Datenbank eingegeben werden kann, um mit bereits bestehenden Quellen verglichen zu werden und dort auch verbleibt, um mit zukünftigen Arbeiten verglichen werden zu können. Dies berechtigt jedoch nicht zur Verwendung oder Vervielfältigung.

Diese Arbeit wurde noch keiner anderen Prüfungsbehörde vorgelegt noch wurde sie bisher veröffentlicht.

.....
Date, Signature

Abstract

Consider this a separate document, although it is submitted together with the rest. The abstract aims at another audience than the rest of the proposal. It is directed at the final decision maker or generalist, who typically is not an expert at all in your field, but more a manager kind of person. Thus, don't go into any technical description in the abstract, but use it to motivate the work and to highlight the importance of your project.

(target size: 15-20 lines)

A large subset of modern software for Kotlin and Kotlin Multiplatform (KMP) uses network interactions extensively. However, standard technologies for organizing network interactions in Kotlin like REST or *Remote Method Invocation* (RMI) require writing a lot of additional code that is usually repetitive across lots of applications. Such code is called *boilerplate code*.

In this thesis we developed a new framework for organizing network interactions in Kotlin and KMP. We called this new framework *Kotlin Remote*. Kotlin Remote is based on ideas of *Remote Procedure Calls* (RPC), generalized version of RMI, and uses implicit parameters to make functions callable on remote machines via network.

We believe that approach we took in Kotlin Remote leads to the great reduction of boilerplate and at the same time makes remote calls explicit for the developer. Furthermore, Kotlin Remote works especially well when server and client share one codebase, which is highly important for KMP environment.

Contents

1	Introduction	1
2	Background and Related work	2
2.1	RPC	2
2.2	Distributed Objects	3
2.3	REST	3
2.4	gRPC	4
2.5	Kotlin RPC	4
2.6	Coeffects	5
2.7	Multi-tier programming languages	6
2.8	Kotlin Ecosystem	6
2.8.1	Kotlin Coroutines	6
2.8.2	Kotlin Multiplatform	7
2.8.3	kotlinx.serialization	7
2.8.4	Ktor	7
2.8.5	Kotlin context parameters	7
3	Kotlin Remote Implementation	8
3.1	Remote functions	8
3.1.1	Client	8
3.1.2	Server	12
3.1.3	Single codebase	13
3.1.4	Remote context relations	14
3.2	Remote functions code generation	14
3.2.1	Remote function body generation	15
3.2.2	Callable map generation	16
3.3	Exception handling	17
3.4	Architecture	18
3.5	Ktor integration	18
3.6	Remote classes	19
3.6.1	Remote classes code generation	20
3.7	Garbage collection	21
4	Evaluation of the Kotlin Remote	21
4.1	Examples	22
4.2	Comparison to alternatives	22
4.2.1	gRPC	22
4.2.2	Kotlin RPC	22
5	Conclusions	22
A	Typesetting Examples	25
A.1	Code Fragments	25
A.2	Tables	25
A.3	Plots	25

1 Introduction

This, like the rest, addresses fellow experts from your field (but not from your particular topic of research). Here you should technically connect to the main concepts from that field and give an outline of your project, stating the research/engineering question that you want to get answered by your project.

(target size: 1-2 pages)

Nowadays, Kotlin programming language is used frequently for developing software that relies heavily on network interactions. Such software includes distributed networks, microservices, client-server applications and much more. Usually code base of all such applications can be roughly separated into two parts: part responsible for actual business logic and part responsible for network interactions. While the former part differs significantly from program to program, network interactions part is usually very similar for a wide variety of applications. This leads to the abundance of boilerplate code in the said part. For convenience from now on we will call part of the program source code responsible for network interactions *network part* of the program. It is desired to make network part of the program as small and simple as possible, because this would allow developers to think less about network and concentrate on the actual business logic.

However, standard ways to implement network part in Kotlin are not well suited for making this part concise. The problem lies in the excessive explicitness of the standard approaches. For example, REST architecture frequently used in Kotlin forces developers to manually write routing logic as well as part of the serialization and deserialization logic. This has its benefits because it provides fine control over the network part logic, but often this fine-tuning capabilities is not needed. Frameworks like gRPC and kRPC are a step further from REST in attempt to decrease boilerplate. But their usage still requires describing and creating RPC services explicitly, arbitrary functions cannot be executed remotely. Other popular approaches like one used in Retrofit framework are basically variations of the named approaches.

The goal of the thesis is to develop alternative approach to organizing network interactions in Kotlin that will make network part smaller and easier by reducing amount of boilerplate code.

To achieve this goal we took a look at RPC in Kotlin from another perspective not common for object-oriented languages. The problem with existing RPC frameworks for Kotlin is their focus on RPC services, special classes which code is generated during compile time using method signatures from user defined interface. Even though this approach is the only option for strict OOP languages (like Java) it can be improved for Kotlin which is not purely OOP language. In this paper we explore using coeffects, a concept from the world of functional programming, to implement RPC framework that require to write much less boilerplate code than more traditional RPC frameworks. More than that, this approach makes network code easily distinguishable from the rest of the program. In Kotlin programming with coeffects can be expressed by using experimental language feature called *context parameters*, which allows function has several receiver parameters, that implicitly passed to the function from the invocation context.

This work has the following structure:

1. Section 2 reviews background and related work on organizing network interaction both in Kotlin ecosystem and outside that.

2. Section 3 describes architecture of the developed approach.
3. Section 4 evaluates the approach by using small applications and comparing them with standard approaches.
4. Section 5 concludes, discuss limitations and future work.

2 Background and Related work

This part should make clear which question, exactly, you are pursuing, and why your project is relevant/interesting. This is the place to explain the background and to review the existing literature. Where does your project extend the state of the art? What weaknesses in known approaches do you hope to overcome? If you have carried out preliminary experiments, describe them here.

(target size: 5-10 pages)

This section will give an overview of RPC as whole, as well as specific frameworks used in Kotlin ecosystem to implement RPC. Also concepts used in this thesis, like *coeffects* and Kotlin context parameters, will be explained here.

Proposed approach is a specific version of RPC, because of that it makes sense to only compare it to other RPC approaches that are used in Kotlin. The most popular ones are gRPC and new Kotlin RPC. Unfortunately, these frameworks make developer writing a substantial amount of boilerplate code, because they require to declare services. Only methods of these services can be executed remotely, but methods of normal classes and top level functions cannot. Basically all the existing RPC frameworks for Kotlin use Remote Method Invocation (RMI), the object-oriented programming analog of RPC, even though Kotlin is not entirely OOP language. The main aim of developed approach is reduction of boilerplate code required to organize network interaction in comparison with existing RPC frameworks in Kotlin.

Apart from RPC frameworks, REST architectural style is also widely used in Kotlin to organize network interactions. Therefore, it is useful to take a brief look at REST as well. Apart from looking at existing frameworks, this section also takes a look on related ideas from functional programming. Namely on *coeffects*, that are foundational concept on which Kotlin Remote is built. Coeffects are tightly related to *implicit parameters*, which Kotlin recently got an implementation of in form of experimental *context parameters* feature. Coeffects also have close ties to *multi-tier programming languages*, in which separation between different machines is encoded in the type system. All of these ideas will be discussed below.

2.1 RPC

Even though the concept of RPC is well known in the community, it is still better to provide some understanding of RPC that is used in this work.

Remote procedure calls (RPC) — a communication protocol which allows an application executing functions on a remote machine (commonly using a shared computer network) as if these functions were executed on the machine where the application is running.

Remote function — function that can be executed on a remote machine.

The goal of RPC is to hide details of network communication from an application and make remote functions calls look like ordinary function calls. This paradigm allows programmer to think less about network details and more about core logic of the application.

The idea of RPC is not new and dates back 1970s [29] with enterprise implementations appearing roughly in the 1980s [1].

It is worth noticing that RPC paradigm has been criticized for trying to hide the complexities of network communication from the developer, which can lead to wrong assumptions about latency and failure modes of remote calls [25]. We are aware of this criticism and as it will be shown later, our framework allows to make remote calls visible at the call site on a type level, while still keeping the boilerplate small.

2.2 Distributed Objects

RPC does not translate well in object-oriented world, where interactions between objects residing on different machines are required. That is why when object-oriented languages became popular specific form of RPC was developed. This form is called *RMI*, and it is a way to build communication between *distributed objects* [30] [26].

Distributed Objects — objects that can be accessed from remote machines.

RMI (remote method invocation) — communication protocol which allows an application executing methods of distributed objects on a remote machine.

Most classic implementations of RMI are CORBA, Java RMI [9] and COM/DCOM. They share lots of architectural similarities and follow the same general idea of providing distributed objects on a type level [20]. However, classic forms of RMI in their fullness are not popular nowadays. These implementations are not used in a modern production code. Modern approaches try to be more lightweight and explicit, they do not try to hide network interactions completely from the developer [23]. Developed framework follows this trend.

2.3 REST

REST (Representational State Transfer) is an architectural style for designing networked applications introduced by Roy Fielding in his PhD thesis [6]. It is built on top of HTTP and uses HTTP methods (GET, POST, PUT, DELETE, etc.) to operate on resources identified by URLs. REST does not specify any particular serialization format, but JSON is the most common one in practice. REST is the most popular way of organizing network interactions in Kotlin today, and is usually implemented using Ktor framework [16] which will be discussed later in section 2.8.4.

In contrast with RPC, REST is centered around resources and not around procedures. From developer's point of view this means that for each new endpoint user has to design URL of the resource, choose HTTP method that should be used to access it, write routing code on the server, write serialization and deserialization logic for request and response bodies and make HTTP request from the client. All of this can be quite tedious especially when client and server share the same codebase, because in this case lots of information that is encoded in URL and HTTP method is essentially duplicated.

REST gives developer a lot of fine-grained control over network interactions. This is beneficial when network communication is exposed to the outside world (for example, public API of a service). However, when network is used for internal communication

between microservices that share the same codebase, this fine-grained control is rarely needed and boilerplate cost dominates.

2.4 gRPC

gRPC [8] is one of the most well-known modern RPC frameworks. It was developed at Google as an open-source successor to internal RPC system called Stubby [23], which was used internally for more than a decade to connect microservices in Google data centers. Today gRPC supports a wide variety of programming languages, including Kotlin.

Communication in gRPC happens over HTTP/2 and uses Protocol Buffers [22] as a default serialization format. Programmer describes services in a special interface definition language (IDL). For each described service gRPC tooling generates client and server stubs in target programming language. Then user implements server stub on the server side and uses client stub on the client side to make remote calls. Stubs here play the same role as stubs in Kotlin RPC, which were briefly mentioned in the section 3.1.1.

Even though gRPC reduces some of the boilerplate compared to manual usage of HTTP frameworks, it still has its drawbacks. Firstly, every remote function should be a method of some service, top level functions cannot be remote. Secondly, separate IDL is required, which means that for each remote function user has to write its description in IDL, implement service method that calls actual function and provide mappings between IDL types and domain types of the application. The latter is especially noticeable when client and server share the same codebase, because in this case types from this shared codebase have to be mapped to and from generated IDL types. Finally, remote call on the client side looks indistinguishable from local call, which is not entirely good, because remote calls are inherently slower and more error-prone than local ones, so developer should be aware of when remote call is happening.

It is worth noticing that gRPC follows RMI approach: every remote function is a method of some service, services are first-class entities. This fits naturally with strict OOP languages like Java, but for Kotlin it is not the only option, as we will show later in this thesis.

2.5 Kotlin RPC

Kotlin RPC (also known as `kotlinx.rpc`) [14] is a relatively new RPC framework from JetBrains targeted specifically at Kotlin and Kotlin Multiplatform. The general idea of Kotlin RPC is similar to gRPC, but it is more idiomatic for Kotlin. Instead of separate IDL, user declares service as a Kotlin interface annotated with `@Rpc`. Compiler plugin then generates client stub for this interface and provides means to expose its implementation on the server side. The exact example of Kotlin RPC usage is shown later in section 3.1.1 on listing 2.

Comparing to gRPC, Kotlin RPC is less verbose for shared-codebase scenarios, because there is no need to write IDL and mapping between IDL types and domain types of the application. However, the same fundamental drawbacks remain: top level functions cannot be remote, every remote function has to be a method of some service and remote calls are not visible at the call site. Just like gRPC, Kotlin RPC is essentially an RMI framework which is not the only viable approach for Kotlin.

2.6 Coeffects

Approach used in Kotlin Remote framework is heavily inspired by ideas from the world of functional programming. Specifically by the concept of *coeffects* [19][3].

Coeffect — property of a computation that describes what this computation requires from its environment in order to be executed.

Coeffects can be thought of as a dual to *effects*. While effects describe what computation does to its environment (for example, mutating a global variable, throwing an exception or performing IO), coeffects describe what computation requires from its environment to be executed (for example, access to specific resource, presence of certain configuration value or, in our case, way to reach remote machine). Petricek et al. [19] introduced unified static analysis for context-dependence that captures several seemingly different examples like liveness analysis, implicit parameters and dataflow caching under one umbrella. Theoretical foundation of coeffects relies on indexed comonads, but in practice coeffects can be expressed using more familiar language constructs.

It is useful to look closer at duality between effects and coeffects. Effects in their modern form are usually expressed via *algebraic effects* [21] and *effect handlers*. Algebraic effects describe operations that computation can perform on its environment (like throwing an exception, reading from a file or sending a network request) and effect handlers describe how exactly these operations should be interpreted. Languages like Koka [17] put algebraic effects in the heart of their type system. The duality is the following: effects describe what computation does to the environment and how this is interpreted, while coeffects describe what computation requires from the environment and how this requirement is satisfied. In the context of this thesis remote call can be seen both as an effect (it performs network IO) and as a coeffect (it requires a way to reach remote machine). In Kotlin Remote we focus on the coeffect side of remote calls, because it allows us to express remote calls as a normal Kotlin functions with implicit context parameter, without introducing any special effect system.

The most well-known practical encodings of coeffects in functional languages are *implicit parameters* [18] and *Reader monad*. Implicit parameters are parameters of the function that are not passed explicitly at the call site, but are resolved automatically from the surrounding context. Reader monad threads read-only environment through computation in a way that does not require explicitly passing it to every function. From the user point of view both mechanisms allow expressing context-dependence of a computation in a type-checked way.

Reader monad is used by RPC framework called Servant [24] for Haskell programming language to implement remote calls. In Servant remote functions return value wrapped into `ClientM` which is basically Reader monad providing remote configuration. Concrete server is provided to remote functions by `runClientM` function. This makes remote calls in Servant clearly remote on a type level, allows top level functions to be remote and there is no need to declare interface for remote functions. Detailed example of Servant usage is presented later in section 3.1.1.

The same principle, when applied to Kotlin, gives us framework that is presented in this thesis. The role of Reader monad or Haskell-style implicit parameters in Kotlin Remote is played by Kotlin *context parameters*, which are described later in section 2.8.5. Context parameter of remote function expresses a coeffect: it states that to execute this function caller has to provide a way to reach remote machine. Such coeffect is visible in the

function signature and, just like in *Servant*, can be satisfied either by configured remote context (and then remote call is performed) or by local context (and then function body is executed locally on the server).

2.7 Multi-tier programming languages

Idea of distinguishing on a type level where particular piece of code is executed (client or server) is not new and was explored by several programming languages designed specifically for web applications. Such languages are often called *multi-tier* or *tierless*, because they allow writing client and server code in the same language and same codebase, while still keeping clear separation between the two on a type level. Multi-tier languages are tightly related to coeffects discussed in the previous section: location of code execution is essentially a coeffect, because computation requires from environment a particular machine to be executed on.

Examples of such languages include *Links* [4], *Ur/Web* [2] and *Opa* [27]. In *Links*, for example, function annotation can specify whether function should be executed on a client or on a server. Compiler then checks that all the calls between client and server functions are done correctly and generates code that handles network communication automatically. *Ur/Web* takes similar approach but additionally provides type-safe SQL queries and HTML templates. *Opa* goes even further by combining client, server and database code in a single language with unified type system.

Approach taken in *Kotlin Remote* can be seen as a library-level approximation of what multi-tier languages do at the language level. Instead of introducing new language with dedicated type system for tiering, we use existing language feature (context parameters) to encode location of code execution on a type level. Of course, this approach is less powerful than full multi-tier language, because it is limited by what can be expressed using context parameters. But it has its own advantages: it can be used in existing *Kotlin* projects without rewriting them in a new language and it integrates naturally with the rest of *Kotlin* ecosystem.

2.8 Kotlin Ecosystem

In the framework developed as part of this thesis many technologies from *Kotlin* Ecosystem are used. Therefore, it makes sense to quickly introduce this technologies.

2.8.1 Kotlin Coroutines

Coroutines [5] is a *Kotlin* feature for asynchronous programming. Functions that can be paused and resumed without blocking the executing thread are called *suspend functions* in *Kotlin* and have a special `suspend` modifier. *Suspend functions* can only be called from other *suspend functions* or from special coroutine builders like `runBlocking` or `launch`. This restriction is sometimes called *function coloring* problem [28][11], because *suspend functions* and ordinary functions are basically two distinct kinds (or *colors*) of functions in *Kotlin*. For our purposes function coloring is actually beneficial, because remote calls are inherently asynchronous, so it is natural for remote functions in *Kotlin Remote* to be *suspend*. Internally coroutines are implemented using continuations. *Suspend functions* are transformed by the *Kotlin* compiler into state machines that take continuation as an additional parameter.

2.8.2 Kotlin Multiplatform

Kotlin Multiplatform (KMP) [13] is a technology that allows writing code that can be compiled to several platforms, including JVM, JavaScript, Native (iOS, Linux, macOS, Windows) and WebAssembly. Kotlin code that targets KMP is split into several so-called *source sets*. There is a common source set with code shared between all the platforms and several platform-specific source sets with code that is specific to particular platform.

KMP introduces `expect/actual` mechanism for declaring code that has different implementation for different platforms. In common source set developer declares function or class with `expect` keyword, this declaration has no body. In each platform-specific source set developer provides actual implementation of this declaration using `actual` keyword. When common source set is compiled for particular platform, `expect` declaration is replaced with corresponding `actual` one.

One important limitation of KMP is that it does not support fully fledged reflection in common source set. JVM target supports rich reflection API, but other targets either do not support reflection at all or support only very limited subset of it. This means that libraries that target KMP can not rely on reflection to gather information about types and functions at runtime. Instead such information has to be gathered at compile time, usually using compiler plugins. Kotlin Remote targets KMP and faces this limitation, which shaped a lot of architectural decisions described later in section 3.2.

2.8.3 kotlinx.serialization

`kotlinx.serialization` [15] is the standard library for serialization in Kotlin. It supports several formats including JSON, CBOR and Protocol Buffers. Serialization logic for user defined classes is generated by Kotlin compiler plugin from class declaration marked with `@Serializable` annotation. This means that serialization works without reflection and as a consequence works in all Kotlin Multiplatform targets. Kotlin Remote uses `kotlinx.serialization` to serialize and deserialize remote function arguments and return values.

2.8.4 Ktor

Ktor [16] is a framework for building asynchronous servers and clients in Kotlin. It is built on top of Kotlin Coroutines and is a part of Kotlin ecosystem maintained by JetBrains. Ktor follows plugin-based architecture: most of the functionality (authentication, content negotiation, logging, etc.) is provided by plugins that user installs explicitly. Kotlin Remote uses Ktor as a default transport: `core-ktor` module provides Ktor plugin called `KRemote` that handles remote calls on the server side and integration of `HttpClient` for the client side. Although Ktor is used as a default transport, the core of Kotlin Remote is transport-agnostic, alternative transports can be plugged in by providing custom implementation of `RemoteClient` interface.

2.8.5 Kotlin context parameters

Context parameters [12] is an experimental Kotlin language feature that introduces implicit parameters to Kotlin. Function with context parameters declares its requirements from the surrounding context using special `context(...)` block before function declaration. Listing 1 shows simple example of function with context parameter.

```

context(logger: Logger)
fun greet(name: String) {
    logger.info("Hello, $name")
}

fun main() {
    val logger = Logger()
    context(logger) {
        greet("World")
    }
}

```

Figure 1: Function with context parameter

In this example function `greet` has context parameter `logger` of type `Logger`. When `greet` is called, `logger` is not passed explicitly, instead it is resolved automatically from the enclosing `context(...)` block. If there is no such block in the lexical scope of the call, compilation error is raised. Context parameters can be of any type and there can be several of them. They are similar to implicit parameters from Scala or Haskell [18], and just like them allow to express effects in Kotlin in a typed way.

In Kotlin Remote context parameters are used to provide remote functions with information about target machine. This way information about remote configuration is propagated through the call chain implicitly, while at the same time being visible in the type signature of remote function. Detailed description of how exactly context parameters are used in Kotlin Remote is provided later in section 3.1.1.

3 Kotlin Remote Implementation

This is the technical core of the thesis. Here you lay out your how you answered your research question, you specify your design of experiments or simulations, point out difficulties that you encountered, etc.

(target size: 5-10 pages)

Framework that was developed in thesis is called *Kotlin Remote*. And from now on we will refer to that this way.

3.1 Remote functions

Function that can be executed on remote machine (aka remote functions) are the integral part of any RPC framework.

In order to reason about remote functions two terms should be introduced: client and server. In RPC world *client* is machine from which remote call is made and *server* is target machine of this remote call, machine where call is actually executed.

3.1.1 Client

Let us take a look at how remote call is done on the client in Kotlin Remote. In order to call a function on a remote machine it is necessary to pass to this function some information

specific to remote call, including address of the target machine and a way to serialize function arguments and deserialize its result. In existing RPC frameworks this information is provided to the function using stub. To better understand what stub is and how it provides necessary information for remote call, let us take a look at the example for Kotlin RPC in the figure 2. User have to create instance of `KtorRpcClient`, which is configured with remote host URL and serialization schema. Then using this client user can create service stub `client.withService<PizzaShop>()`, where `PizzaShop` is interface which implementation is generated automatically by Kotlin RPC. Such an interface is usually called *service* which remote machine provides.

```
@Rpc
interface PizzaShop { suspend fun orderPizza(pizza: Pizza): Receipt }
fun main() = runBlocking {
    val client: KtorRpcClient = HttpClient { ... }.rpc {
        url { host = "localhost" }
        serialization { json() }
    }

    val pizzaShop: PizzaShop = client.withService<PizzaShop>()

    pizzaShop.orderPizza(Pizza("Pepperoni"))
}
```

Figure 2: Remote function call in Kotlin RPC

Stub `pizzaShop` use client with which stub was created to make remote calls. It has some drawbacks:

1. Top level functions can not be called using this approach
2. User have to declare interface just for remote functions
3. When calling methods of the stub user does not know that calls results in network interaction, it is completely implicit
4. When user wants to call remote functions of the same service on several target machines, several stubs have to be created

gRPC uses virtually the same approach as Kotlin RPC but a bit more cumbersome.

But stubs is not the only way to provide remote function with information it needs. Alternative approach is used in functional programming. Let us take a look at remote calls in RPC framework for Haskell programming language named Servant [24].

Here `position` and `hello` are remote functions. We can see their return type is `ClientM x`, where `x` is the actual value function compute and `ClientM` is a monad. Inside `queries` function several calls to remote functions are made. The place where information about target machine is provided is actually `runClientM queries ... "localhost"`. `ClientM` is basically a Reader monad, because it provides configuration for remote functions to run.

This functional RPC approach solves some of the problems of OOP approach that uses stubs. Top level functions now can be called, there is no need to declare interface, all the remote functions are clearly remote on a type level and there is no need to create

```

position :: Int -> Int -> ClientM Position
hello :: Maybe String -> ClientM HelloMessage

queries :: ClientM (Position, HelloMessage)
queries = do
    pos <- position 10 10
    message <- hello (Just "servant")
    return (pos, message)

run :: IO ()
run = do
    Right (pos, message) <- runClientM queries ... "localhost"
    print pos
    print message

```

Figure 3: Remote function call in Servant

several stubs to call the same functions on several target machines. Resolution of these problems are important for Kotlin because Kotlin is not solely OOP language, it has a lot of attributes of functional and procedure languages. So we believe RPC framework for Kotlin will benefit greatly by solving stated problems.

There is an alternative to Reader monad applicable to Kotlin that is called *implicit parameters*. These are parameters of the function that are resolved from the function invocation context and should not be passed explicitly when function is called in contrast to normal parameters. New language feature for Kotlin called *context parameters* [12] is basically implementation of implicit parameters for Kotlin. Using this feature it is possible to develop RPC framework for Kotlin that uses functional approach to make remote calls. That is exactly what we did in Kotlin Remote.

Let us imagine we want to execute some function multiply remotely. Here is client implementation for it in the figure 4.

```

fun pass(): Nothing = error("pass")
fun multiply(lhs: Long, rhs: Long): Long { pass() }

```

Figure 4: Client part of multiply function

Body of the function is basically undefined, because it is not needed on client. To pass to multiply function a way to communicate with remote machine, we can add context parameter to multiply, like on figure 5. Reader may notice that it is inconvenient to write `c.client.call(...)` for every remote function. This problem will be addressed later in section 3.2 when code generation done by the framework will be discussed. On listing 5 remote client contains all the information needed for remote call, like target address and serialization schema. Default implementation for `RemoteClient` uses *Ktor* [16] `HttpClient`, which is setup by the user. In order to call multiply user can write code from figure 6.

The setup of `HttpClient` is quite standard, the interesting part is `remoteSerializersModule` function. This function declares serialization schema for remote function parameters and result. It returns instance of `SerializationModule` from `kotlinx.serialization` [15]. This `SerializationModule` has serializers to serialize and deserialize remote func-

```

interface RemoteConfig {
    val client: RemoteClient
}

context(c: RemoteConfig)
suspend fun multiply(lhs: Long, rhs: Long): Long {
    return c.client.call<Long>(
        RemoteCall("multiply", arrayOf(lhs, rhs))
    )
}

```

Figure 5: Multiply function with context

```

data object CalculatorRemoteConfig : RemoteConfig {
    override val client: RemoteClient = HttpClient {
        defaultRequest { url("http://localhost:8080") ... }
        install(ContentNegotiation) {
            json(Json {
                serializersModule = remoteSerializersModule {
                    callableMap = CallableMap(...)
                }
            })
        }
    }.remoteClient("/call")
}

fun main() = runBlocking {
    context(CalculatorRemoteConfig) {
        println(multiply(6, 1))
    }
}

```

Figure 6: Remote function call

tion parameters and return values. `RemoteClient.call` accepts `RemoteCall` from figure 7. Field `callableName` is the name of the function which is called and `arguments` is arguments with which this function is called. This class has a custom serializer that serializes arguments: `Array<Any?>` using `callableName`. This serializer is created by `remoteSerializersModule`. To implement this custom serializer information about remote function with name `callableName` is needed. The only way to get this information in Kotlin Multiplatform (*KMP*) is by asking user to provide it or gathering it automatically somehow. The problem is this information cannot be gathered at runtime because KMP (unlike Java or Kotlin JVM) does not support fully fledged reflection. This means that we cannot get function signature at runtime or type by its name, etc. But this is exactly information needed for serialization. For now, we will think that required information is gathered somehow and stored inside structure called `CallableMap`. This is basically a map from fully qualified name of remote function to information about this function which is represented by `RemoteCallable` from figure 7.

The pipeline is the following: `remoteSerializersModule` creates serializer for `RemoteCall` using `CallableMap`. This serializer uses `CallableMap` and `RemoteCall.callableName` to

determine types of parameters in `RemoteCall.arguments` and serialize them. On the server `RemoteCall` is deserialized in a similar manner. Server implementation will be discussed in details in the next section.

```
class CallableMap(val callableMap: Map<String, RemoteCallable> = mapOf())

class RemoteCall(
    val callableName: String,
    val arguments: Array<Any?>,
)

data class RemoteCallable(
    val returnType: KType,
    val parameterTypes: Array<KType>,
)
```

Figure 7: RemoteCall and RemoteCallable class

The simplest way to get `CallableMap` it is to ask user to write it manually. Let us think for now that user has to provide information about all the remote functions by describing `CallableMap`. This problem will be addressed later in section 3.2.2.

3.1.2 Server

It was described how to use remote functions from the client side. Now it is time to understand how to handle remote calls on the server side. When remote function is called from the client HTTP request is sent to the server. For example from figure 6 HTTP will be sent to address with URL `"http://localhost:8080/call"` there will be no query parameters and payload will be serialized version of `RemoteCall`. For this specific example it will be in JSON format. Here it is worth noticing that even though the framework was written with HTTP in mind, any transport protocol could be used. To do that alternative implementation for `RemoteClient` interface should be provided.

User could write handler for such a request himself. Either using Ktor or anything else. But to make this process easier, framework provides default handlers for remote calls. To use it Ktor server with installed `KRemote` plugin should be created. This can be done by writing code from figure 8. This will run HTTP server on URL `"http://localhost:8080/call"`. Just like in client `CallableMap` is used to describe remote functions this server can handle.

The problem is that server as opposed to client should not only serialize and deserialize function arguments and return value, but also invoke function itself. To do that `RemoteCallable` should be extended with field of type `RemoteInvokator` (figure 9). `RemoteInvokator` is basically a reference to the remote method. It is easier to understand by example on figure 10 with `RemoteCallable` for multiply function. Of course, this multiply function should have a normal body and no context parameters as opposed to the client version.

Now server can invoke `RemoteInvokator` when it receives `RemoteCall`. To do that server finds `RemoteCallable` in `CallableMap` by `callableName` from `RemoteCall`.


```

fun main() {
    embeddedServer(Netty, port = 8080) {
        install(CallLogging)
        install(KRemote) {
            callableMap = CallableMap(..)
        }
        routing {
            remote("/call")
        }
    }.start(wait = true)
}

```

Figure 8: Kotlin Remote server

```

data class RemoteCallable(
    val returnType: RemoteType,
    val invokator: RemoteInvokator,
    val parameters: Array<RemoteParameter>,
)

fun interface RemoteInvokator {
    suspend fun call(parameters: Array<Any?>): Any?
}

```

Figure 9: RemoteCallable with invokator

```

RemoteCallable(
    returnType = typeOf<Long>(),
    invokator = { multiply(it[0] as Long, it[1] as Long) },
    parameters = arrayOf(typeOf<Long>(), typeOf<Long>()),
)

```

Figure 10: RemoteCallable for multiply

3.1.3 Single codebase

Described methods for creating client and server are not very good, when client and server share the same codebase. By that we mean that client and server code is in a single Gradle module. Gradle [7] is the most popular build system used by Kotlin, Kotlin does not have its own concept of modules. The idea is that using the same function for client calls and server handling is more convenient than using two functions with the same signature. However, bodies of the remote functions are different for client point of view and server point of view. In order to fix that we can use more elaborate context parameters. Basically a "switch" between server and client body of the remote function is needed, and such switch can be encoded using the same context parameter that was used for remote call. Take a look at modified version of multiply function in figure 11. RemoteContext is basically a sum of types, it is either LocalContext or ConfiguredContext. Notice how covariant type parameter T and type Nothing are used here. Because Nothing is a bottom type in Kotlin type system (in other words it is subtype of every type) and T is covariant, LocalContext is a subtype RemoteContext for all T. It means that any remote function can be called in a LocalContext, and it will type check. When remote function is called in

a `LocalContext` it will be executed in a current coroutine and will not result in a request to the server. Because of that the same function can be called from server in a local context and from client but in configured context.

```
sealed interface RemoteContext <out T: RemoteConfig>
object LocalContext: RemoteContext<Nothing>
class ConfiguredContext<T: RemoteConfig>(val config: T): RemoteContext<T>

context(ctx: RemoteContext<RemoteConfig>)
suspend fun multiply(lhs: Long, rhs: Long) =
    if (ctx is LocalContext) {
        lhs * rhs
    } else {
        (ctx as ConfiguredContext<RemoteConfig>).config.client.call<Long>(
            RemoteCall("multiply", arrayOf(lhs, rhs))
        )
    }
}
```

Figure 11: Updated multiply

Now `RemoteInvoker` should be changed a bit to reflect this duality of remote functions. Added context parameter hints that remote function can be only called locally on a server. Now server have to provide `LocalContext` for all the invoker calls.

```
fun interface RemoteInvoker {
    context(_: LocalContext)
    suspend fun call(parameters: Array<Any?>): Any?
}
```

Figure 12: Updated RemoteInvoker

3.1.4 Remote context relations

Contexts of remote functions can be used to express quite complex relations between capabilities of different servers using subtyping. For example let us imagine that there are two servers for arithmetic operations. One server only provides basic operations like multiplication etc., while other also supports trigonometric functions. That can be expressed on a type level in the way shown on figure 13. Clients here form a hierarchy. Notice how `mul` and `divide` can be called from `sin` function. More than that, calls to `mul` and `divide` from `sin` will not result in a remote call, because everything will be done in `LocalContext` provided by the invoker on the server. However, if user really wants to call functions marked with `ArithmeticConfig` from inside of `sin` remotely, he can change context by calling `context(ArithmeticConfigImpl) { ... }`.

3.2 Remote functions code generation

Some of the actions required from user in order to use remote functions can be delegated to the compiler using code generation. Namely, `CallableMap` and calls to contexts of remote functions can be generated by the compiler. There are several standard ways to generate code in Kotlin. Kotlin Remote uses the most powerful one: compiler plugins.

```

interface ArithmeticConfig: RemoteConfig
object ArithmeticConfigImpl: ArithmeticConfig { ... } // http://localhost:8001
object TrigonometricConfig: ArithmeticConfig { ... } // http://localhost:8002
context(_: RemoteContext<ArithmeticConfig>)
suspend infix fun Double.mul(rhs: Double) = this * rhs
context(_: RemoteContext<ArithmeticConfig>)
suspend infix fun Double.divide(rhs: Double) = this / rhs
context(_: RemoteContext<TrigonometricConfig>)
suspend fun sin(x: Double): Double {
    var (term, sum, n) = Triple(x, x, 1.0)
    while (abs(term) > 1e-5) {
        val divisor = (2.0 mul n) mul (2.0 mul n + 1.0)
        term = term mul -1.0 mul x mul x divide divisor
        sum = sum + term
        n++
    }
    return sum
}

```

Figure 13: Hierarchy of clients

Kotlin compiler architecture consists of two main parts: backend and frontend. Kotlin compiler plugins can also be separated into two categories: frontend plugins and backend plugins. Frontend plugins allow extending behavior of Kotlin compiler frontend. Backend plugins do the same thing but with compiler's backend. Both compiler's frontend and backend consists of the series of so-called lowerings, transformations of internal representation (IR) of the Kotlin source code in compiler. Plugins allow adding new lowerings, some of the lowerings do not change IR and only checks for some invariants.

The main difference between frontend and backend plugins is that frontend plugins only allow adding new definitions and change attributes of the old ones, but they do not allow changing bodies of definitions. However, results of plugins' work on frontend can be seen inside IDE, making frontend a great place to add verification, like checking that all the remote functions are suspend. Some code generation can also be done on frontend easier than on backend. Because of that Kotlin Remote generate code on both frontend and backend. It is also worth noticing that backend of the Kotlin compiler have less stable API. So it is generally a good idea to make changes on frontend whenever it is possible.

Nevertheless, Kotlin Remote plugin generates almost all the code in backend plugins, because they are much more powerful. This is why from now on we will focus mainly on backend part of the Kotlin Remote compiler plugin.

3.2.1 Remote function body generation

Firstly, there is a small problem with using compiler plugin. Now it is hard to distinguish remote functions from normal ones. Of course, context parameter can be used to do that, but it is a bad approach because not every function with context parameter of type `RemoteContext` should be remote. Sometimes we want to add context parameter to a function to mark that this function can call remote functions. But it shouldn't mean that the function itself is remote. Let us take a look at example on figure 14. Implementation of `multiply` is from figure 11. Here `power` function has a context parameter of type

RemoteContext but is not itself remote, its body is executed locally, but context parameter shows that power function is capable of calling remote functions like multiply. Basically context parameter on power function signals that it has a *coeffect*, in other words it requires from environment a way to make remote calls.

To express such coeffects we need a separate way to mark remote functions. In Kotlin Remote annotation Remote is used for that. On figure 14 you can notice that multiply function has such annotation. Every function marked with Remote undergoes a body transformation that is exactly the same with transformation of multiply function that was done manually on figure 11. Name of remote call is a fully qualified name of the function. It is worth noticing that function marked with @Remote annotation has to have context parameter of type RemoteContext, otherwise error will be raised by Kotlin Remote frontend compiler plugin. This error will be visible in IntelliJ IDEA.

```
@Remote
context(_: RemoteContext<RemoteConfig>)
suspend fun multiply(lhs: Long, rhs: Long) = ...

context(_: RemoteContext<RemoteConfig>)
private suspend fun power(base: Long, power: Int): Long {
    return generateSequence { base }
        .take(power)
        .fold(1L) { acc, x -> multiply(acc, x) }
}
```

Figure 14: Local function with RemoteContext

3.2.2 Callable map generation

In the end of section 3.1.1 we said that there is a problem with user having manually describe CallableMap. Kotlin Remote addresses this issue by automatically generating callable map using function called genCallableMap from figure 15. As you can see this function basically has no body. This is because every call to this function is substituted with CallableMap instance generated by compiler plugin. One can say that genCallableMap is basically a macro (like in C). But a very smart macro. Kotlin Remote compiler plugin scan the source code during compilation, finds each function marked with @Remote and for each such function generates entry in CallableMap describing this function. All the information needed is accessible during compile time. This means that callable map generated genCallableMap can have lots and lots of entries, because functions from all over the source code are gathered.

It is worth noticing that genCallableMap adds to callable map even private, protected and internal functions. This means that by using genCallableMap it is possible to invoke such functions effectively breaking encapsulation. So it is recommended to keep all the remote functions public. Another quirk about genCallableMap is that it will not generate entries from remote functions from dependencies, because source code of dependencies is not accessible (it is already compiled). To alleviate that dependencies can save their callable maps generated by genCallableMap, for example, in a public static property. This way dependent code can use these saved callable maps by, for example, merging them with its own callable map.

```
fun genCallableMap(): CallableMap =
    throw IllegalStateException("Intrinsic function was called directly")
```

Figure 15: genCallableMap function

3.3 Exception handling

During executing of remote functions on remote machines exceptions can occur. Such erroneous situations should be handled by the client that made remote call. Here a tough decision should be made. We basically have two options: hide the fact that exception occurred on remote machine or make it obvious. Both options have its benefits and drawbacks, but it was decided to opt for the second approach, because we wanted to make calls to remote functions more seamless. So in the current implementation exception thrown on remote machine can be caught on the client as if function was called locally. Take a look at listing 16. This code will output exception from listing 1. You can see that there is now frames in exception related to remote call except synthetic stack frame == Remote Call Boundary == that indicates a boundary between stack frames from remote machine and local frames.

```
@Remote
context(_: RemoteContext<RemoteConfig>)
suspend fun div(x: Int, y: Int) = x / y

fun main() {
    try {
        ServerConfig.runWith { div(10, 0) }
    } catch (_: ArithmeticException) {
        e.printStackTrace()
    }
}
```

Figure 16: Remote exception handling

```
java.lang.ArithmeticException: / by zero
    at proposalExamples.ExceptionKt.div(Exception.kt:11)
    at == Remote Call Boundary ==.(Unknown Source)
    at proposalExamples.ExceptionKt.main(Exception.kt:15)
    ...
```

Listing 1: Remote exception content

To implement this at first remote function should be able to return either a result or an error. In Kotlin this can be expressed like on listing 2.

```
sealed interface RemoteResponse<T> {
    data class Success<T>(val value: T): RemoteResponse<T>
    data class Failure(val error: Throwable): RemoteResponse<Nothing>
}
```

Listing 2: Return type of remote functions

However, two more things should be implemented: serialization of exceptions and altering stack traces of exceptions. Unfortunately, exceptions are not serialized by default using

`kotlinx.serialization`, a library that Kotlin Remote uses for serialization. To make exceptions serializable they should be registered as polymorphic subclasses of `Throwable`. This is already done for all exceptions from a standard Kotlin Multiplatform library. When user wants to throw custom exception, he should register it in a `SerializersModule` using `throwableSerializer` function like at listing 3. When exception is not registered it will be serialized as `UnregisteredRemoteException`. To alter stack traces in KMP expect/actual functions are used. Unfortunately only JVM backend allows to change exceptions. For all other backends exception from remote machine is wrapped in a `RemoteException` as a cause.

```
SerializersModule {
    polymorphic(Throwable::class) {
        subclass(MyException::class, throwableSerializer(::MyException))
    }
}
```

Listing 3: Custom exception registration

3.4 Architecture

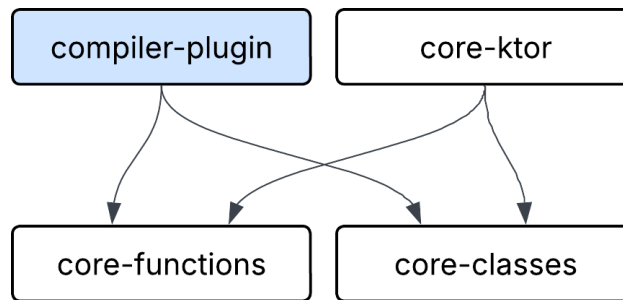


Figure 17: Module dependency graph

Now is a good time to take a look at architecture of kotlin-remote framework. At 17 is shown module dependency graph of the framework. Each rectangle is a Gradle module. Before we only described `compiler-plugin` and `core-functions` module. But there are two more modules that extend functionality of the mentioned ones. `core-classes` contain support for distributed objects and `core-ktor` provides an implementation of the transport protocol for Kotlin Remote using Ktor.

3.5 Ktor integration

Kotlin Remote is meant to be transport agnostic. This is done by splitting code into three modules: `core-functions`, `core-classes`, `core-ktor`. Until now, we almost exclusively have been discussing `core-functions` module that contains logic related to remote functions. Now let us briefly discuss `core-ktor` module which contains Kotlin Remote integration with Ktor. Ktor is a mature framework that offers lots of functions like auth support, logging etc. Kotlin Remote used together with Ktor supports the same functionality Ktor does. For example, authentication can be supported using code from listing 4. Notice call to `handleRemoteCall` function in server. This function passes control to `core-ktor` module that deserializes arguments of remote call, invokes remote function, serializes and sends result to the client. Inside `/call` route `handleRemoteCall` is called implicitly.

```

val server = embeddedServer(Netty, port = 8080) {
    install(Authentication) {
        basic("auth-basic") { validate { ... } }
    }
    install(KRemote) { callableMap = genCallableMap() }
    routing {
        authenticate("auth-basic") {
            post("/callAuth") {
                println("Welcome to the protected route!")
                handleRemoteCall()
            }
        }
        remote("/call")
    }
}

data object AuthServerConfig : RemoteConfig {
    override val client: RemoteClient = HttpClient {
        ...
        install(Auth) { ... }
    }.remoteClient(genCallableMap(), "/callAuth")
}

```

Listing 4: Authentication support

3.6 Remote classes

Because Kotlin is not only procedural and functional language but also relies heavily on OOP, it was decided to add support to call methods of the classes remotely, not only top-level functions. Basic support for that is achieved easily. The only important difference between top level functions and method function is that methods have implicit `this` parameter. In our case we can think about `this` parameter just like about any other regular parameter. So calling a remote method serializes `this` and other method parameters on client, sends call on sever, server deserializes `this` and other parameters and calls method on deserialized `this`. The good news are this is already supported without any additional effort because Kotlin compiler treats `this` as a regular parameter of the function. But there are two huge problems with this approach. Firstly, `this` should be serializable, and serialization logic should be provided by the user. Secondly, methods that mutate `this` will mutate deserialized copy of `this`, which basically means that they will have no effect on the original `this`.

All these problems are not new and were addressed in lots of existing RMI frameworks, like Java RMI. In Kotlin Remote we decided to use well-known approach to solve these problems: stubs. Classes in Kotlin Remote can be marked with `@RemoteSerializable` annotation. For each remote serializable class custom `kotlinx.serialization` serializer is generated.

This custom serializer serializes remote class as stub. Stub is a subclass of remote class that is generated for every remote class. Stub only has two fields: ID and URL of the machine it was created on. During serialization of remote class it is added to the special map called `RemoteInstancesPool`. Keys in this pool are stub IDs and values are original

classes.

Stub is deserialized either as original class in case it is deserialized on the machine it was serialized on and `RemoteInstancesPool` contains it, or as just stub. To understand whether stub was serialized on a certain machine, URL of this machine is compared with URL from the stub.

This URL is only used for this disambiguation of the nodes and garbage collection (more about it later in 3.7). This URL is not used to make remote calls at all, address of the target machine is still received from context parameter of remote functions. URL is specified on the server when application is configured. It means that user should know address of the machine the server is running on.

Basically stubs are "remote references" that are supported by Kotlin Remote without altering logic related to remote function. Only serialization functionality is changed. Because of that logic related to remote classes lives in a separate Gradle module named `core-classes` which neither depends nor is dependent on `core-functions` module described earlier. It is worth noticing that stubs are serialized as is, this allows to pass remote references from one machine to another even if reference is created on a third machine.

3.6.1 Remote classes code generation

It was said that for each `@RemoteSerializable` class a stub subclass is generated. This subclass is generated inside original class like on listing 5. `RemoteClassStub` extends `Calculator` and implements interface `Stub`. Stub interface can be used by user to check whether a certain instance of `Calculator` is a stub or not. Notice how `RemoteClassStub` invokes `Calculator`'s default constructor even though `Calculator` doesn't have one. This is achieved by generating default constructor for `Calculator` in Kotlin Remote backend compiler plugin. `Calculator` is also made open implicitly by the same plugin on backend. Because these changes are made on backend, `Calculator` class cannot be extended, and its default constructor cannot be called from user defined Kotlin code. Only code generated by compiler plugin can do that.

```
@RemoteSerializable
class Calculator(private var init: Int) {
    @Remote
    context(_: RemoteContext<RemoteConfig>)
    suspend fun multiply(x: Int): Int {
        init *= x
        return init
    }
    class RemoteClassStub(
        override val id: Long,
        override val url: String
    ) : Calculator(), Stub
}
```

Listing 5: Remote class stub

Unfortunately, without reflection (and we cannot use it because of KMP) it is unclear how to deserialize `Calculator` as its `RemoteClassStub` because every remote class has its

own `RemoteClassStub` type. To solve this issue another Kotlin Remote intrinsic is used, called `genRemoteClassList`. Take a look at listing 6. List of `RemoteClassDescriptor` is used to generate serializers and deserializers for remote classes using contextual serialization from `kotlinx.serialization`.

```
data class RemoteClassDescriptor<T : Any>(
    val clazz: KClass<T>,
    val serialName: String,
    val stubFabric: (Long, String) -> T,
)

fun genRemoteClassList(): List<RemoteClassDescriptor<Any>> = RemoteIntrinsic
```

Listing 6: `genRemoteClassList` intrinsic

3.7 Garbage collection

Now there is no way to remove remote classes from `RemoteInstancesPool`. This is a serious resource leak. To resolve this issue several techniques can be used, including reference counting or full-blown distributed garbage collection. But we decided to opt for leasing approach, because it is easy, lightweight and durable. Each time stub is deserialized on a machine this machine begins sending lease requests to machine where original class of the stub is stored. To determine address of the machine where original class is stored `url` field of the stub is used. When machine with original class does not get lease requests for specific class for a long time it removes this class from `RemoteInstancesPool`. Machine owning a stub stops sending lease requests when this stub is garbage collected on the owner machine. To implement that weak references are used. These are references that are not taken into account to mark reachable objects during garbage collection. This is basically the whole algorithm. The beauty of this approach is that when client owning a stub crashes, server is able to remove original class from its pool gracefully. Garbage collection is enabled by default, but can be disabled and fine-tuned when needed. Ktor integration provides transport for leases, user can provide its own ways to deliver leases.

4 Evaluation of the Kotlin Remote

This section discusses criteria that are used to evaluate the research results. Make sure your results can be used to published research results, i.e., to the already known state-of-the-art.

(target size: 5-10 pages)

4.1 Examples

4.2 Comparison to alternatives

4.2.1 gRPC

4.2.2 Kotlin RPC

5 Conclusions

Summarize the main aspects and results of the research project. Provide an answer to the research questions stated earlier.

(target size: 1/2 page)

References

- [1] Andrew D. Birrell and Bruce Jay Nelson. “Implementing remote procedure calls”. In: *ACM Trans. Comput. Syst.* 2.1 (Feb. 1984), pp. 39–59. ISSN: 0734-2071. DOI: [10.1145/2080.357392](https://doi.org/10.1145/2080.357392). URL: <https://dl.acm.org/doi/10.1145/2080.357392> (visited on 02/07/2026).
- [2] Adam Chlipala. “Ur/Web: a simple model for programming the web”. In: *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL ’15. New York, NY, USA: Association for Computing Machinery, Jan. 2015, pp. 153–165. ISBN: 978-1-4503-3300-9. DOI: [10.1145/2676726.2677004](https://doi.org/10.1145/2676726.2677004). URL: <https://doi.org/10.1145/2676726.2677004>.
- [3] *Coeffects: Context-aware programming languages*. URL: <https://tomasz.net/coeffects/> (visited on 10/15/2025).
- [4] Ezra Cooper et al. “Links: web programming without tiers”. In: *Proceedings of the 5th international conference on Formal methods for components and objects*. FMCO’06. Berlin, Heidelberg: Springer-Verlag, 2007, pp. 266–296. ISBN: 978-3-540-74791-8. DOI: [10.1007/978-3-540-74792-5_12](https://doi.org/10.1007/978-3-540-74792-5_12). URL: https://doi.org/10.1007/978-3-540-74792-5_12.
- [5] *Coroutines / Kotlin*. en. URL: <https://kotlinlang.org/docs/coroutines-overview.html> (visited on 04/15/2026).
- [6] Roy Thomas Fielding. “Architectural styles and the design of network-based software architectures”. AAI9980887. PhD thesis. University of California, Irvine, 2000.
- [7] *Gradle Build Tool*. en-US. Feb. 2026. URL: <https://gradle.org/> (visited on 02/13/2026).
- [8] *gRPC*. en. URL: <https://grpc.io/> (visited on 02/07/2026).
- [9] *Java Development Kit Version 17 API Specification*. en. URL: <https://docs.oracle.com/en/java/javase/17/docs/specs/rmi/index.html> (visited on 02/10/2026).
- [10] *KEEP/notes/0004-multishot-continuations.md at main · Kotlin/KEEP*. en. URL: <https://github.com/Kotlin/KEEP/blob/main/notes/0004-multishot-continuations.md> (visited on 10/15/2025).
- [11] *KEEP/notes/code-coloring.md at master · Kotlin/KEEP*. URL: <https://github.com/Kotlin/KEEP/blob/master/notes/code-coloring.md> (visited on 10/15/2025).
- [12] *KEEP/proposals/KEEP-0367-context-parameters.md at main · Kotlin/KEEP*. URL: <https://github.com/Kotlin/KEEP/blob/main/proposals/KEEP-0367-context-parameters.md> (visited on 10/15/2025).
- [13] *Kotlin Multiplatform*. en. URL: <https://kotlinlang.org/docs/multiplatform.html> (visited on 04/15/2026).
- [14] *Kotlin/kotlinx.rpc*. URL: <https://github.com/Kotlin/kotlinx.rpc> (visited on 04/15/2026).
- [15] *Kotlin/kotlinx.serialization*. original-date: 2017-07-20T11:25:23Z. Feb. 2026. URL: <https://github.com/Kotlin/kotlinx.serialization> (visited on 02/11/2026).
- [16] *Ktor: Build Asynchronous Servers and Clients in Kotlin*. en. URL: <https://ktor.io/> (visited on 02/11/2026).

- [17] Daan Leijen. *Koka: a Functional Language with Effects*. URL: <https://koka-lang.github.io/> (visited on 04/15/2026).
- [18] Jeffrey R. Lewis et al. "Implicit parameters: dynamic scoping with static types". In: *Proceedings of the 27th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. POPL '00. New York, NY, USA: Association for Computing Machinery, Jan. 2000, pp. 108–118. ISBN: 978-1-58113-125-3. DOI: [10.1145/325694.325708](https://doi.org/10.1145/325694.325708). URL: <https://doi.org/10.1145/325694.325708> (visited on 04/15/2026).
- [19] Tomas Petricek, Dominic Orchard, and Alan Mycroft. "Coeffects: unified static analysis of context-dependence". In: *Proceedings of the 40th international conference on Automata, Languages, and Programming - Volume Part II*. ICALP'13. Berlin, Heidelberg: Springer-Verlag, July 2013, pp. 385–397. ISBN: 978-3-642-39211-5. DOI: [10.1007/978-3-642-39212-2_35](https://doi.org/10.1007/978-3-642-39212-2_35). URL: https://doi.org/10.1007/978-3-642-39212-2_35 (visited on 02/09/2026).
- [20] František Plášil and Michael Stal. "An architectural view of distributed objects and components in CORBA, Java RMI and COM/DCOM". en. In: *Software - Concepts & Tools* 19.1 (Mar. 1998), pp. 14–28. ISSN: 1432-2188. DOI: [10.1007/s003780050004](https://doi.org/10.1007/s003780050004). URL: <https://doi.org/10.1007/s003780050004> (visited on 02/07/2026).
- [21] Gordon Plotkin and Matija Pretnar. "Handlers of algebraic effects". In: *Proceedings of the 18th European Symposium on Programming Languages and Systems: Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2009*. ESOP '09. Berlin, Heidelberg: Springer-Verlag, Mar. 2009, pp. 80–94. ISBN: 978-3-642-00589-3. DOI: [10.1007/978-3-642-00590-9_7](https://doi.org/10.1007/978-3-642-00590-9_7). URL: https://doi.org/10.1007/978-3-642-00590-9_7.
- [22] *Protocol Buffers*. en. URL: <https://protobuf.dev/> (visited on 04/15/2026).
- [23] Louis Ryan|. *gRPC Motivation and Design Principles*. en. Section: blog. Sept. 2015. URL: <https://grpc.io/blog/principles/> (visited on 02/10/2026).
- [24] *Servant - haskell-servant*. URL: <https://www.servant.dev/> (visited on 02/08/2026).
- [25] Andrew S Tanenbaum and Robbert van Renesse. "A Critique of the Remote Procedure Call Paradigm". en. In: ().
- [26] Andrew S. Tanenbaum and Maarten van Steen. *Distributed Systems: Principles and Paradigms (2nd Edition)*. USA: Prentice-Hall, Inc., Sept. 2006. ISBN: 978-0-13-239227-3.
- [27] *The Opa Language*. URL: <http://opalang.org/> (visited on 10/15/2025).
- [28] *What Color is Your Function? – journal.stuffwithstuff.com*. URL: <https://journal.stuffwithstuff.com/2015/02/01/what-color-is-your-function/> (visited on 10/15/2025).
- [29] James E. White. "A high-level framework for network-based resource sharing". In: *Proceedings of the June 7-10, 1976, national computer conference and exposition*. AFIPS '76. New York, NY, USA: Association for Computing Machinery, June 1976, pp. 561–570. ISBN: 978-1-4503-7917-5. DOI: [10.1145/1499799.1499878](https://dl.acm.org/doi/10.1145/1499799.1499878). URL: <https://dl.acm.org/doi/10.1145/1499799.1499878> (visited on 02/10/2026).
- [30] Ann Wollrath, Roger Riggs, and Jim Waldo. "A distributed object model for the javaTM system". In: *Proceedings of the 2nd conference on USENIX Conference on Object-Oriented Technologies (COOTS) - Volume 2*. COOTS'96. USA: USENIX Association, June 1996, p. 17. (Visited on 02/07/2026).

A Typesetting Examples

A.1 Code Fragments

```
fn num_to_ordinal_expr(x: u32) -> String {  
    format!("{}-{}", x, match (x % 10, x % 100) {  
        (1, 1) | (1, 21...91) => "st",  
        (2, 2) | (2, 22...92) => "nd",  
        (3, 3) | (3, 23...93) => "rd",  
        _ => "th"  
    })  
}
```

A.2 Tables

Number	Description
7	A lucky number in Western culture
8	A lucky number in Chinese and other Asian cultures
42	Answer to the ultimate question of life, the universe, and everything
404	Not found

Table 1: Useless insights I gained with no further meaning

A.3 Plots

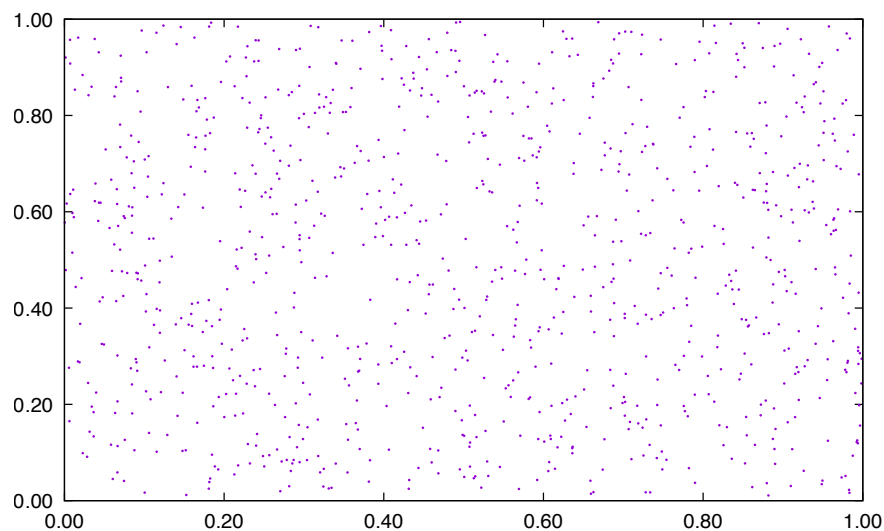


Figure 18: Many dots distributed over a two dimensional unit space without any discernible pattern or deeper meaning